LAPORAN PRAKTIKUM PEKAN 4

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 4 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Queue*.

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar struktur data serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 28 April 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB 1 1

PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB 2 2

PEMBAHASAN 2

2.1 *Queue* 2

2.2 Class QueueArray 3

2.3 Class QueueArrayDriver 5

2.4 Class QueueLinkedList 7

2.5 Class ReverseData 8

2.6 Class IterasiQueue 10

2.7 Tugas Praktikum Pekan 4 11

BAB 3 21

KESIMPULAN 21

DAFTAR PUSTAKA 22

LAMPIRAN 23

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, struktur data memegang peranan penting dalam mengelola dan mengorganisir data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah *Queue*. *Queue* merupakan struktur data linear yang mengikuti prinsip *First In First Out* (FIFO), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Konsep ini mirip dengan antrian dalam kehidupan sehari- hari, seperti antrian di loket atau kasir, di mana orang yang datang lebih dahulu akan dilayani terlebih dahulu.

Pemahaman mengenai *queue* sangat penting karena banyak diterapkan dalam berbagai sistem komputer, seperti manajemen proses pada sistem operasi, antrian printer, hingga pengelolaan data pada jaringan komputer. Selain itu, *queue* juga menjadi dasar dalam implementasi algoritma tertentu, seperti *breadth-first search* (BFS) pada graf. Dengan memahami konsep *queue*, mahasiswa dapat mengembangkan kemampuan dalam merancang solusi yang lebih efisien untuk berbagai permasalahan pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar struktur data *queue*.
2. Mengetahui cara implementasi *queue* dalam bahasa pemrograman Java.
3. Mengetahui dan memahami operasi-operasi dasar pada *queue* seperti *enqueue, dequeue, peek, isEmpty*.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai konsep dasar struktur data, khususnya *queue*.
2. Meningkatkan kemampuan pemograman dalam bahasa Java.
3. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2
PEMBAHASAN

2.1 *Queue*

Queue merupakan salah satu struktur data linear yang memiliki prinsip kerja *Firat In First Out* (FIFO), yaitu elemen yang pertama kali dimasukkan akan menjadi elemen pertama yang dikeluarkan. Konsep ini sangat mirip dengan antrian dalam kehidupan sehari-hari. Dalam implementasinya di bahasa pemrograman Java, *queue* dipresentasikan sebagai sebuah *interface* yang berada dalam paket java.util dan merupakan bagian dari *Java Collections Framework*.

Pada struktur data *queue*, terdapat dua bagian utama, yaitu *front* dan *rear*. Elemen baru ditambahkan di bagian belakang melalui operasi *enqueue*, sedangkan penghapusan elemen dilakukan dari bagian depan melalui operasi *dequeue*. Dengan mekanisme ini, urutan data tetap terjaga sesuai dengan waktu masuknya data. Selain itu, *queue* juga menyediakan operasi tambahan seperti *peek* untuk melihat elemen terdepan tanpa menghapusnya, serta *isEmpty* untuk mengecek apakah *queue* kosong.

Dalam Java, operasi *queue* diimplementasikan melalui beberapa method penting seperti add() atau offer() untuk menambahkan elemen, serta remove() atau poll() untuk mengambil dan menghapus elemen. Penggunaan method offer() dan poll() umumnya lebih disarankan karena tidak menimbulkan *exception* ketika *queue* kosong atau penuh, sehingga lebih aman digunakan dalam program.

Queue memiliki banyak peranan dalam dunia nyata, terutama dalam sistem komputer. Contohnya adalah pengaturan proses pada sistem operasi, antrian pada printer, serta pengelolaan request pada server. Selain itu, *queue* juga digunakan dalam algoritma seperti *Breadth-First Search* (BFS) pada struktur data graf. Dengan adanya *queue*, proses pengolahan data dapat dilakukan secara terstruktur dan efisien karena mengikuti urutan yang jelas.

Implementasi *queue* dalam Java dapat dilakukan menggunakan berbagai struktur data, seperti *array* maupun *linked list*. Salah satu implementasi yang paling umum adalah menggunakan class LinkedList yang sudah mengimplementasikan *interface queue*. Dengan demikian, programmer tidak perlu membuat struktur dari

awal, melainkan cukup memanfaatkan fitur, yang telah tersedia dalam Java. Hal ini membuat pengembangan program menjadi lebih cepat, efisien, dan terstruktur.

2.2 Class QueueArray

Berikut kode program Java mengenai class QueueArray :

```
public class QueueArray_2511532027 {
	int front_2027, rear_2027, size_2027;
	int capacity_2027;
	int array_2027[];

	public QueueArray_2511532027(int capacity_2027) {
		this.capacity_2027 = capacity_2027;
		front_2027 = this.size_2027 = 0;
		rear_2027 = capacity_2027 - 1;
		array_2027 = new int[this.capacity_2027];
	}

	boolean isFull_2027(QueueArray_2511532027 queue_2027) {
		return (queue_2027.size_2027 == queue_2027.capacity_2027);
	}

	boolean isEmpty_2027(QueueArray_2511532027 queue_2027) {
		return (queue_2027.size_2027 == 0);
	}

	void enqueue_2027(int item_2027) {
		if (isFull_2027(this))
			return;
		this.rear_2027 = (this.rear_2027 + 1) % this.capacity_2027;
		this.array_2027[this.rear_2027] = item_2027;
		this.size_2027 = this.size_2027 + 1;
		System.out.println(item_2027 + " enqueued to queue");
	}

	int dequeue_2027() {
		if (isEmpty_2027(this))
			return Integer.MIN_VALUE;
		int item_2027 = this.array_2027[this.front_2027];
```

```
            this.front_2027 = (this.front_2027 + 1) %
this.capacity_2027;
            this.size_2027 = this.size_2027 - 1;
            return item_2027;
      }
      int front_2027() {
            if (isEmpty_2027(this))
                  return Integer.MIN_VALUE;
            return this.array_2027[this.front_2027];
      }
      int rear_2027() {
            if (isEmpty_2027(this))
                  return Integer.MIN_VALUE;
            return this.array_2027[this.rear_2027];
      }
      // mencetak elemen antrian
      void display_2027() {
            int i_2027;
            if (front_2027 == rear_2027) {
                  System.out.printf("\nAntrian Kosong\n");
                  return;
            }
            // kunjungi dari belakang dan cetak
            for (i_2027 = front_2027; i_2027 < rear_2027; i_2027++) {
                  System.out.printf(" %d <--", array_2027[i_2027]);
            }
                  return;
      }
}
```

Program di atas merupakan implementasi struktur data *queue* menggunakan *array* dalam bahasa pemrograman Java. Kelas QueueArray_2511532027 memiliki atribut front_2027, rear_2027, size_2027, capacity_2027, dan array_2027. Variabel front_2027 digunakan untuk menunjuk elemen paling depan dalam antrian, sedangkan rear_2027 menunjuk elemen paling belakang. Variabel size_2027 menyimpan jumlah elemen yang ada dalam *queue*, dan capacity_2027 menunjukkan kapasitas maksimum *queue*. *Array* array_2027[] digunakan sebagai media penyimpanan data dalam *queue*. Pada konstruktor, kapasitas *queue*

diinisialisasi sesuai parameter yang diberikan, kemudian nilai front dan size diatur ke 0, serta rear diatur ke capacity – 1. Hal ini bertujuan untuk mempersiapkan mekanisme *circular queue*.

Program ini juga memiliki beberapa method penting untuk mendukung operasi *queue*. Method isFull_2027() digunakan untuk mengecek apakah *queue* sudah penuh dengan cara membandingkan ukuran saat ini dengan kapasitas maksimum. Sementara itu, method isEmpty_2027() digunakan untuk mengecek apakah *queue* kosong. Kedua method ini sangat penting untuk mencegah terjadinya kesalahan saat melakukan operasi penambahan atau penghapusan data.

Operasi utama dalam *queue* diimplementasikan melalui method enqueue_2027() dan dequeue_2027(). Method enqueue_2027(int item_2027) berfungsi untuk menambahkan elemen ke dalam *queue*. Sebelum menambahkan, program akan mengecek apakah *queue* penuh. Jika tidak penuh, maka posisi rear akan dgeser menggunakan operasi modulo ((rear + 1) % capacity) agar dapat berfungsi sebagai *circular queue*. Data kemudian dimasukkan ke dalam *array* pada posisi rear, dan ukuran *queue* ditambah satu. Sebaliknya, method dequeue_2027() digunakan untuk menghapus elemen dari *queue*. Jika *queue* kosong, maka method akan mengembalikan nilai Integer.MIN_VALUE sebagai penanda. Jika tidak kosong, data pada posisi front akan diambil, kemudian posisi front digeser menggunakan operasi modulo, dan ukuran *queue* dikurangi satu.

Selain itu, terdapat method front_2027() dan rear_2027() yang masing-masing digunakan untuk melihat elemen terdepan dan elemen terakhir tanpa menghapusnya. Kedua method ini juga melakukan pengecekan apakah *queue* kosong sebelum mengembalikan nilai. Jika *queue* kosong, maka akan dikembalikan nilai Integer.MIN_VALUE.

Terakhir, method display_2027() digunakan untuk menampilkan isi antrian. Method ini akan mencetak elemen dari posisi front hingga rear. Jika kondisi front sama dengan rear, maka program akan menampilkan, pesan bahwa antrian kosong. Perulangan digunakan untuk menampilkan setiap elemen dalam *queue* secara berurutan.

2.3 Class QueueArrayDriver

Berikut kode program Java mengenai class QueueArrayDriver :

```java
public class QueueArrayDriver_2511532027 {
	public static void main (String[] args) {
		QueueArray_2511532027 queue_2027 = new
QueueArray_2511532027(1000);
		queue_2027.enqueue_2027(10);
		queue_2027.enqueue_2027(20);
		queue_2027.enqueue_2027(30);
		queue_2027.enqueue_2027(40);
		System.out.println("Item di depan " +
queue_2027.front_2027());
		System.out.println("Item paling belakang " +
queue_2027.rear_2027());
		System.out.println("tampilan queue");
		queue_2027.display_2027();
		System.out.println();
		System.out.println(queue_2027.dequeue_2027() + " dihapus
dari queue");
		System.out.println("Item di depan : " +
queue_2027.front_2027());
		System.out.println("Item paling belakang: " +
queue_2027.rear_2027());
		System.out.println("tampilan queue setelah satu data
dihapus");
		queue_2027.display_2027();
	}
}
```

Program di atas merupakan kelas driver yang digunakan untuk menguji implementasi struktur data *queue* yang telah dibuat pada kelas QueueArray_2511532027. Pada method main, program dimulai dengan membuat sebuah objek *queue* bernama queue_2027 dengan kapasitas sebesar 1000 elemen. Inisialisasi ini bertujuan untuk menyediakan ruang penyimpanan data dalam bentuk *array* yang akan digunakan sebagai antrian. Setelah objek *queue* dibuat, program menambahkan beberapa elemen ke dalam *queue* menggunakan method enqueue_2027(). Proses ini menunujukkan bagaimana operasi *enqueue* bekerja, yaitu menambahkan elemen ke bagian belakang antrian.

Selanjutnya, program menampilkan elemen yang berbeda di posisi depan dan belakang *queue* menggunakan method front_2027() dan rear_2027(). Informasi ini dicetak ke layar untuk menunjukkan kondisi terkini dari *queue* setelah dilakukan beberapa operasi penambahan data. Kemudian, method display_2027() dipanggil untuk menampilkan seluruh isi antrian, sehingga pengguna dapat melihat urutan elemen yang tersimpan di dalam *queue* dari depan ke belakang.

Program ini juga mendemonstrasikan operasi *dequeue* dengan memanggil method dequeue_2027(), yang berfungsi untuk menghapus elemen dari bagian depan *queue*. Elemen yang dihapus kemudian ditampilkan ke layar sebagai informasi bahwa data tersebut telah keluar dari antrian. Setelah proses penghapusan dilakukan, program kembali menampilkan elemen depan dan belakang *queue* untuk memperlihatkan perubahan yang terjadi. Terakhir, method display_2027() dipanggil kembali untuk menampilkan isi *queue* setelah satu elemen dihapus.

2.4 Class QueueLinkedList

Berikut kode program Java mengenai class QueueLinkedList :

```
import java.util.*;

public class QueueLinkedList_2511532027 {
	public static void main (String[] args) {
		Queue<Integer> q_2027 = new LinkedList<>();
		// tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
		for (int i_2027 = 0; i_2027 < 6; i_2027++)
			q_2027.add(i_2027);
		// Menampilkan isi antrian
		System.out.println("Elemen Antrian " + q_2027);
		// Untuk menghapus kepala antrian
		int hapus_2027 = q_2027.remove();
		System.out.println("Hapus elemen = " + hapus_2027);
		System.out.println(q_2027);
		// Untuk melihat antrian terdepan
		int depan_2027 = q_2027.peek();
		System.out.println("Kepala Antrian = " + depan_2027);

		int banyak_2027 = q_2027.size();
		System.out.println("Size Antrian = " + banyak_2027);
```

```
        }
}
```

Program di atas merupakan implementasi struktur data *queue* menggunakan fasilitas yang telah disediakan oleh Java melalu *Java Collections Framework*. Pada program ini digunakan *interface* Queue dengan implementasi kelas LinkedList. Hal ini memungkinkan pembuatan *queue* tanpa harus membangun struktur daat dari awal, karena LinkedList sudah menyediakan berbagai method yang mendukung operasi *queue*. Pada method main, program dimulai dengan membuat objek *queue* bernama q_2027 yang bertipa Queue<Integer>, sehingga hanya dapat menyimpan data bertipe integer.

Selanjutnya, program melakukan proses penambahan elemen ke dalam *queue* menggunakan perulangan for. Nilai yang dimasukkan adalah 0 hingga 5, sehingga terdapat enam elemen dalam antrian. Penambahan elemen dilakukan menggunakan method add(), yang secara otomatis menempatkan elemen di bagian belakang antrian. Setelah proses penambahan selesai, seluruh isi *queue* ditampilkan menggunakan System.out.println, yang secara langsung menampilkan isi antrian karena LinkedList memiliki representasi string bawaan.

Program kemudian mendemonstrasikan operasi penghapusan elemen dengan menggunakan method remove(). Method ini akan mengambil sekaligus menghapus elemen yang berada di bagian depan *queue*, sesua dengan prinsip FIFO. Nilai yang dihapus disimpan dalam variabel hapus_2027 dan ditampilkan ke layar. Setelah itu, isi *queue* kembali ditampilkan untuk menunjukkan bahwa elemen pertama telah berhasil dihapus dari antrian.

Selain itu, program juga menggunakan method peek() untuk melihat elemen yang berada di bagian depan *queue* tanpa menghapusnya. Nilai tersebut disimpan dalam variabel depan_2027 dan ditampilkan ke layar sebagai informasi kepala antrian saat ini. Terakhir, program menggunakan method size() untuk mengetahui jumlah elemen yang tersisa dalam *queue* setelah dilakukan operasi penghapusan. Nilai ini disimpan dalam variabel banyak_2027 dan ditampilkan ke layar.

2.5 Class ReverseData

Berikut kode program Java mengenai class ReverseData :

```
import java.util.LinkedList;
import java.util.Queue;
```

```java
import java.util.Stack;

public class ReverseData_2511532027 {
	public static void main (String[] args) {
		Queue<Integer> q_2027 = new LinkedList<Integer>();
		q_2027.add(1);
		q_2027.add(2);
		q_2027.add(3); // [1, 2, 3]
		System.out.println("sebelum reverse" + q_2027);
		Stack<Integer> s_2027 = new Stack<Integer>();
		while (!q_2027.isEmpty()) { // Q -> S
			s_2027.push(q_2027.remove());
		}
		while (!s_2027.isEmpty()) { // S -> Q
			q_2027.add(s_2027.pop());
		}
		System.out.println("sesudah reverse = " + q_2027); // [3,
2, 1]
	}
}
```

Program di atas bertujuan untuk membalik urutan data dalam sebuah *queue* dengan memanfaatkan struktur data tambahan berupa *stack*. Pada method main, program membuat objek *queue* q_2027 menggunakan implementasi LinkedList. Kemudian, beberapa elemen dimasukkan ke dalam *queue*, yaitu 1, 2, dan 3, sehingga urutan awal antrian adalah [1, 2, 3]. Sebelum proses pembalikan dilakukan, isi *queue* ditampilkan ke layar untuk menunjukkan kondisi awal data.

Selanjutnya, program membuat objek *stack* s_2027 yang digunakan sebagai media bantu untuk membalik urutan data. Proses pembalikan dimulai dengan memindahkan seluruh elemen dari *queue* ke dalam *stack* menggunakan perulangan while. Selama *queue* tidak kosong, elemen pada bagian depan *queue* diambil menggunakan method remove() kemudian dimasukkan ke dalam *stack* menggunakan method push(). Proses ini menyebabkan urutan data di dalam *stack* menjadi terbalik, karena *stack* bekerja dengan prinsip *Last In First Out* (LIFO).

Setelah semua elemen berpindah ke dalam *stack*, langkah berikutnya adalah memindahkan kembali elemen dari *stack* ke *queue*. Proses ini juga menggunakan

perulangan while, di mana selama *stack* tidak kosong, elemen diambil dari *stack* menggunakan method pop() kemudian dimasukkan ke *queue* menggunakan method add(). Karena sifat *stack* yang LIFO, elemen yang terakhir masuk akan keluar terlebih dahulu, sehinnga urutan data yang dimasukkan kembali ke *queue* menjadi terbalik dari urutan awal. Pada akhir program, isi *queue* kembali ditampilkan untuk menunjukkan hasil dari proses pembalikan.

2.6 Class IterasiQueue

Berikut kode program Java mengenai class IterasiQueue :

```
import java.util.Iterator;
import java.util.*;

public class IterasiQueue_2511532027 {
	public static void main (String[] args) {
		Queue<String> q_2027 = new LinkedList<>();
		q_2027.add("Praktikum");
		q_2027.add("Struktur");
		q_2027.add("Data");
		q_2027.add("Dan");
		q_2027.add("Algoritma");
		Iterator<String> iterator_2027 = q_2027.iterator();
		while (iterator_2027.hasNext()) {
			System.out.print(iterator_2027.next() + " ");
		}
	}
}
```

Program di atas merupakan contoh implementasi struktur data *queue* yang dikombinasikan dengan proses iterasi menggunakan *iterator* pada bahasa pemrograman Java. Pada method main, program dimulai dengan membuat objek *queue* bernama q_2027 yang diimplementasikan menggunakan kelas LinkedList. *Queue* ini bertipe data String, sehingga digunakan untuk menyimpan sekumpulan kata. Selanjutnya, program menambahkan beberapa elemen ke dalam *queue* menggunakan method add(), yaitu “Praktikum”, “Struktur”, “Data”, “Dan”, dan “Algoritma”. Penambahan elemen ini mengikuti prinsip *enqueue*, di mana setiap elemen baru akan ditempatkan di bagian belakang antrian.

Setelah *queue* terisi, program menggunakan objek Iterator untuk melakukan penelusuran terhadap seluruh elemen dalam *queue*. Iterator dibuat dengan memanggil method iterator() dari objek *queue*, yang kemudian disimpan dalam variabel iterator_2027. Proses iterasi dilakukan menggunakan perulangan while dengan kondisi iterator_2027.hasNext(), yang berfungsi untuk mengecek apakah masih terdapat elemen berikutnya dalam *queue*. Jika masih ada, maka elemen tersebut diambil menggunakan method next() dan langsung ditampilkan ke layar menggunakan System.out.print().

Melalui penggunaan *iterator*, program dapat mengakses setiap elemen dalam *queue* secara berurutan tanpa mengubah isi atau struktur antrian tersebut. Hal ini berbeda dengan operasi seperti remove() yang dapat menghapus elemen dari *queue*. Dengan demikian, iterasi sangat berguna ketika hanya ingin membaca atau menampilkan data tanpa memodifikasi isi *queue*. Hasil dari program ini adalah menampilkan seluruh elemen *queue* dalam satu baris sesuai urutan penyimpanannya, yaitu “Praktikum Struktur Data Dan Algoritma”.

2.7 Tugas Praktikum Pekan 4

Berikut tugas praktikum yang diberikan :

TUGAS QUEUE

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan **struktur data Queue (Antrian)** menggunakan **array dalam bahasa Java**.

Studi kasus yang digunakan adalah **antrian pelanggan di loket pelayanan**. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip **FIFO (First In First Out)**.

Program harus mampu:

- Menambahkan pelanggan ke antrian (**enqueue**)
- Menghapus pelanggan dari antrian (**dequeue**)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

2. Aturan Penamaan (Wajib Dipatuhi)

1. **Nama Kelas**

   Gunakan akhiran **NIM lengkap**

   Contoh:

- `AntrianLoket_2311532008`
- `Queue_2311532008`

1. **Nama File Java**

   Harus sama dengan nama class

   Contoh:
   `AntrianLoket_2311532008.java`

2. **Penamaan Variabel (Gunakan standar berikut):**

- `front` → indeks depan

Gambar 2.1

- `rear` → indeks belakang
- `max` → kapasitas array
- `queue[]` → array penyimpan data

---

3. Struktur Tugas

Program minimal memiliki:

a. Deklarasi dan Inisialisasi

- Array untuk menyimpan data antrian
- Variabel `front` dan `rear`
- Kapasitas maksimal queue

---

b. Method yang Wajib Ada

1. **enqueue()**
   - Menambahkan data ke belakang antrian
   - Jika penuh → tampilkan pesan
2. **dequeue()**
   - Menghapus data dari depan antrian
   - Jika kosong → tampilkan pesan
3. **display()**
   - Menampilkan seluruh isi antrian
4. **isEmpty()**
   - Mengembalikan nilai true jika kosong
5. **isFull()**
   - Mengembalikan nilai true jika penuh
6. **Reverse()**
   - Membalik seluruh isi antrian

Gambar 2.2

c. Menu Program

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
```

4. Contoh Output

```
=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

Pilih menu: 4
Isi antrian:
1. Budi
2. Andi

Pilih menu: 2
Budi telah dilayani

Pilih menu: 3
Isi antrian:
1. Andi
```

Gambar 2.3

```
Pilih menu: 5
Program selesai
```

Gambar 2.4

Berikut kode program Java dari tugas praktikum :

```java
public class QueueAntrianLoket_2511532027 {
    int front_2027, rear_2027, max_2027;
    String queue_2027[];

    public QueueAntrianLoket_2511532027(int max_2027) {
```

```
        this.max_2027 = max_2027;
        queue_2027 = new String[max_2027];
        front_2027 = -1;
        rear_2027 = -1;
    }

    public boolean isEmpty_2027() {
        return front_2027 == -1;
    }

    public boolean isFull_2027() {
        return rear_2027 == max_2027 - 1;
    }

    public void enqueue_2027(String data_2027) {
        if (isFull_2027()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_2027()) {
                front_2027 = 0;
            }
            rear_2027++;
            queue_2027[rear_2027] = data_2027;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    public void dequeue_2027() {
        if (isEmpty_2027()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue_2027[front_2027] + " telah
dilayani");
            if (front_2027 == rear_2027) {
                front_2027 = rear_2027 = -1;
            } else {
                front_2027++;
            }
```

```
        }
    }

    public void display_2027() {
        if (isEmpty_2027()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no_2027 = 1;
            for (int i_2027 = front_2027; i_2027 <= rear_2027; i_2027++)
{
                System.out.println(no_2027 + ". " + queue_2027[i_2027]);
                no_2027++;
            }
        }
    }

    public void reverse_2027() {
        if (isEmpty_2027()) {
            System.out.println("Antrian kosong!");
        } else {
            int i_2027 = front_2027;
            int j_2027 = rear_2027;

            while (i_2027 < j_2027) {
                String temp_2027 = queue_2027[i_2027];
                queue_2027[i_2027] = queue_2027[j_2027];
                queue_2027[j_2027] = temp_2027;
                i_2027++;
                j_2027--;
            }

            System.out.println("Isi antrian:");
            int no_2027 = 1;
            for (int k_2027 = front_2027; k_2027 <= rear_2027; k_2027++)
{
                System.out.println(no_2027 + ". " + queue_2027[k_2027]);
                no_2027++;
```

```
            }
        }
    }
}
```

Program di atas merupakan implementasi struktur data *queue* yang disimulasikan sebagai antrian pada loket pelayanan menggunakan *array bertipe* String. Kelas ini memiliki atribut front_2027 sebagai penunjuk elemen terdepan, raear_2027 sebagai penunjuk elemen terakhir, max_2027 sebagai kapasitas maksimum antrian, serta queue_2027[] sebagai *array* untuk menyimpan data. Pada konstruktor, nilai kapasitas diinisialisasi sesuai parameter yang diberikan, kemudian *array* dibuat dengan ukuran tersebut. Nilai awal front_2027 dan rear_2027 diatur menjadi -1 yang menandakan bahwa antrian masih kosong.

Program menyediakan method isEmpty_2027() dan isFull_2027() untuk mengecek kondisi antrian. Method isEmpty_2027() akan mengembalikan nilai benar jika front bernilai -1, sedangkan isFull_2027() akan bernilai benar jika rear telah mencapai indeks maksimum *array* (max – 1). Kedua method ini sangat penting untuk mencegah kesalahan saat melakukan operasi penambahan maupun penghapusan datadalam *queue*.

Operasi penambahan data diimplementasikan pada method enqueue_2027(String data_2027). Jika antrian penuh, program akan menampilkan pesan “Antrian penuh!”. Namun jika masih tersedia ruang, maka data akan ditambahkan ke posisi rear. Jika sebelumnya antrian kosong, maka front diatur ke 0 terlebih dahulu. Setelah itu, nilai rear ditambah satu dan data dimasukkan ke dalam *array*. Program juga memberikan pesan bahwa data berhasil ditambahkan ke antrian. Sebaliknya, operasi penghapusan data dilakukan melalui method dequeue_2027(). Jika antrian kosong, maka akan ditampilkan pesan “Antrian kosong!”. Jika tidak, data pada posisi front akan dianggap telah dilayani dan ditampilkan ke layar. Jika setelah penghapusanantrian menjadi kosong (front sama dengan rear), maka kedua indeks direset menjadi -1, jika tidak maka front akan digeser ke indeks berikutnya.

Method display_2027() digunakan untuk menampilkan seluruh isi antrian. Jika antrian kosong, maka akan ditampilkan pesan yang sesuai. Namun jika terdapat data, program akan menampilkan setiap elemen secara berurutan dari front hingga

rear, disertai nomor urut untuk mempermudah pembacaan. Hal ini menunjukkan kondisi aktual dari antrian seperti pada sistem loket pelayanan.

Selain itu, terdapat method reverse_2027() yang berfungsi untuk membalik urutan elemen dalam antrian. Method ini bekerja dengan menggunakan dua variabel indeks, yaitu i_2027 yang dimulai dari depan dan j_2027 dari belakang. Kedua indeks akan saling bertukar nilai elemen hingga bertemu di tengah, sehingga urutan data menjadi terbalik. Setelah proses pembalikan selesai, isi antrian ditampilkan kembali untuk menunjukkan hasil perubahan.

```
import java.util.Scanner;

public class QueueAntrianLoketDriver_2511532027 {
    public static void main(String[] args) {
        Scanner input_2027 = new Scanner(System.in);
        QueueAntrianLoket_2511532027 antrian_2027 = new
QueueAntrianLoket_2511532027(1000);

        int pilihan_2027;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_2027 = input_2027.nextInt();
            input_2027.nextLine();

            switch (pilihan_2027) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama_2027 = input_2027.nextLine();
                    antrian_2027.enqueue_2027(nama_2027);
                    break;

                case 2:
                    antrian_2027.dequeue_2027();
```

```
                    break;

                case 3:
                    antrian_2027.display_2027();
                    break;

                case 4:
                    antrian_2027.reverse_2027();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_2027 != 5);

        input_2027.close();
    }
}
```

Program di atas merupakan kelas driver yang digunakan untuk menjalankan dan menguji sistem antrian loket yang telah dibuat pada kelas QueueAntrianLoket_2511532027. Program ini bersifat interaktif karena menggunakan kelas Scanner untuk menerima input dari pengguna melalui keyboard. Pada bagian awal method main, program membuat objek Scanner untuk membaca input, serta objek antrian_2027 dengan kapasitas 1000 data yang akan digunakan untuk menyimpan antrian pelanggan.

Selanjutnya, program menggunakan struktur perulangan do-while untuk menampilkan menu secara berulang hingga pengguna memilih untuk keluar. Menu yang ditampilkan terdiri dari menambah antrian, menghapus antrian, menampilkan antrian, membalik urutan antrian, dan keluar dari program. Pengguna diminta untuk memasukkan pilihan, yang kemudian disimpan dalam variabel pilihan _2027.

Setelah itu, program menggunakan struktur switch-case untuk menjalankan perintah sesuai dengan pilihan pengguna.

Pada pilihan pertama, program meminta pengguna memasukkan nama pelanggan kemudian memanggil method enqueue_2027 untuk menambahkan data tersebut ke dalam antrian. Pilihan kedua digunakan untuk menghapus antrian dengan memanggil method dequeue_2027(), yang akan melayani pelanggan di posisi terdepan. Pilihan ketiga akan menampilkan seluruh isi antrian melalui method display_2027(), sehingga pengguna dapat melihat urutan pelanggan yang sedang menunggu. Pilihan keempat digunakan untuk membalik urutan antrian dengan memanggil method reverse_2027(), sedangkan pilihan kelima akan menghentikan program.

Program juga dilengkapi dengan penanganan kesalahan sederhana melalui bagian default pada switch-case, yang akan menampilkan pesan jika pengguna memasukkan pilihan yang tidak valid. Perulangan akan terus berjalan selama pengguna tidak memilih opsi kelima.Setelah program selesai, objek Scanner ditutup menggunakan method close() untuk menghindari kebocoran.

BAB 3
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *queue* merupakan salah satu struktur data liner yang bekerja berdasarkan prinsip *First In First Out* (FIFO), di mana data yang pertama kali masuk akan menjadi data pertama yang keluar. Implementasi *queue* dapat dilakukan menggunakan berbagai cara, seperti *array* maupun *linked list*, serta dapat memanfaatkan fitur bawaan Java melalui *Java Collection Framework*. Melalui serangkaian program yang telah dibuat dapat dipahami bahwa *queue* memiliki peran penting dalam pengelolaan data yang terstruktur dan efisien. Selain itu, penggunaan operasi dasar menunjukkan bahwa *queue* dapat diterapkan dalam berbagai permasalahn nyata di bidang komputer.

Sebagai saran, diharapkan mahasiswa untuk lebih mendalami konsep struktur data lainnya yang berkaitan dengan *queue*, seperti *deque* (*double-ended queue*) atau *priority queue*, serta memahami perbedaan performa dari setiap implementasi. Selain itu, mahasiswa juga dapat mencoba mengembangkan program yang lebih kompleks dan realistis. Penting juga untuk memperhatikan penanganan kesalahan agar program menjadi lebih aman dan stabil. Dengan pelatihan yang lebih intensif dan eksplorasi lebih lanjut, diharapkan pemahaman terhadap struktur data *queue* dan penerapannya dalam dunia pemrograman akan semakin meningkat.

DAFTAR PUSTAKA

[1] https://www.callicoder.com/java-queue/ [Diakses 28 April 2026]

[2] https://www.javaspring.net/blog/java-queue-example/ [Diakses 28 April 2026]

[3] https://www.delftstack.com/howto/java/enqueue-and-dequeue-java/ [Diakses 28 April 2026]

LAMPIRAN

Gambar 2.1 12

Gambar 2.2 13

Gambar 2.3 14

Gambar 2.4 14